

LINQ ElementAt and ElementAtOrDefault Method in C#

In this article, I will discuss **LINQ ElementAt and ElementAtOrDefault Methods in C#** with Examples. Please read our previous article discussing the basics of [LINQ Element Operators](#). The ElementAt and ElementAtOrDefault Methods belong to the Elements Operator Category. So, before proceeding further, let us first understand what Element Methods or Operators are in LINQ.

What are LINQ Element Methods or Operators?

The LINQ Element Methods or Operators are used to return a single element from a data source using the element's index or a predicate, i.e., a condition. These Element Operators can be used with a single data source or on a query of multiple data sources.

In LINQ, the ElementAt and ElementAtOrDefault methods are used to retrieve an element from a specific index in a sequence. They behave similarly to array indexing but can be used with any type that implements `IEnumerable<T>`.

LINQ ElementAt Method:

The LINQ ElementAt Method in C# returns the element at a specified index in a sequence. If the data source is empty or the provided index value is out of range, we will get `ArgumentOutOfRangeException`. If the Data Source is Null, then it will throw `ArgumentNullException`. If you go to the definition of the ElementAt method, then you will see the following signature.

```
public static TSource ElementAt<TSource>(this IEnumerable<TSource> source,
int index);
```

As you can see, this method takes one parameter, i.e., the index position. Then, it will return the element present in that index position of the data source. There is no overloaded version available for this method.

- **Purpose:** Returns the element at a specified index in the sequence.
- **Usage:** You use ElementAt when you are confident that the index you specify is within the bounds of the collection. If the specified index is out of range, it will throw an `ArgumentOutOfRangeException`.

Example to Understand LINQ ElementAt Method in C#.

Let us see an example to Understand the LINQ ElementAt Method in C# with both Method and Query Syntax. Our requirement is to fetch the Element Present in Index Position 1 using the ElementAt Method. For a better understanding, please have a look at the following example. Here, we have created one data source that contains integer numbers. Then, we fetch the element present in index position 1 by using the ElementAt method, and to that method, we pass the value 1. There is no such operator called ElementAt available to write the Query Syntax. If you want, you can combine the method syntax and query syntax to write the code, as shown in the example below.

```
using System.Linq;
using System;
```

```

using System.Collections.Generic;
namespace LINQElementAtDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Using ElementAt Method
            //Fetch the Element from Index Position 1 using Method Syntax
            //ElementAt Method returns a Single Value
            int MethodSyntax = numbers.ElementAt(1);
            //Query Syntax
            int QuerySyntax = (from num in numbers
                               select num).ElementAt(1);
            //Printing the value returned by the ElementAt Method
            Console.WriteLine(MethodSyntax);
            Console.ReadLine();
        }
    }
}

```

Run the application, and it will print 2 in the output window as in index position 1; the value 2 is there.

What happens if the Index Value is out of the range of the collection?

If we are passing the Index value to the ElementAt method, which is out of the Range of the collection, or if we pass a negative value to the ElementAt method, then the ElementAt method will throw `ArgumentOutOfRangeException`. That means it will throw `ArgumentOutOfRangeException` when the index is less than 0 or greater than or equal to the number of elements in the source. Let us understand this with an example. Please have a look at the following example. The data source or collection contains 10 elements so that the index will start from 0 to 9. Let's see what happens when we try to fetch the element from index position 10 or pass a negative index value, as shown in the example below.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQElementAtDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Using ElementAt Method
            //Fetch the Element from Index Position -1 or 10 using Method Syntax
            //int MethodSyntax = numbers.ElementAt(-1);
            int MethodSyntax = numbers.ElementAt(10);
        }
    }
}

```

```

        //Printing the value returned by the ElementAt Method
        Console.WriteLine(MethodSyntax);
        Console.ReadLine();
    }
}

```

Output:

```

//Data Source
List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };

//Using ElementAt Method
//Fetch the Element from Index Position -1 or 10 using Method Syntax
//int MethodSyntax = numbers.ElementAt(-1);
int MethodSyntax = numbers.ElementAt(10);

```

Exception Unhandled

System.ArgumentOutOfRangeException: 'Index was out of range. Must be non-negative and less than the size of the collection. Parameter name: index'

[View Details](#) | [Copy Details](#)

▸ Exception Settings

ElementAt Method

What happens If we call the LINQ ElementAt method on an Empty Data Source in C#?

If we call the LINQ ElementAt method on an Empty Data Source in C#, it will throw **ArgumentOutOfRangeException**. Let us understand this with an example. Please have a look at the following example. Here, the data source or collection is Empty, and on the Empty Data Source, we call the LINQ ElementAt Method.

```

using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQElementAtDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source is Empty
            List<int> numbers = new List<int>();
            //Using ElementAt Method
            int MethodSyntax = numbers.ElementAt(1);
        }
    }
}

```

```

        //Printing the value returned by the ElementAt Method
        Console.WriteLine(MethodSyntax);
        Console.ReadLine();
    }
}

```

Output:

```

//Data Source is Empty
List<int> numbers = new List<int>();

//Using ElementAt Method
int MethodSyntax = numbers.ElementAt(1);

```



Exception Unhandled

System.ArgumentOutOfRangeException: 'Index was out of range. Must be non-negative and less than the size of the collection. Parameter name: index'

[View Details](#) | [Copy Details](#)

▸ Exception Settings

ElementAt Method

What happens If we call the LINQ ElementAt method on a Data Source, which is Null?

If we call the LINQ ElementAt method on a Data Source that is Null, then it will throw ArgumentNullException. Let us understand this with an example. Please have a look at the following example. Here, the data source or collection is Null; on the Null Data Source, we call the LINQ ElementAt Method.

```

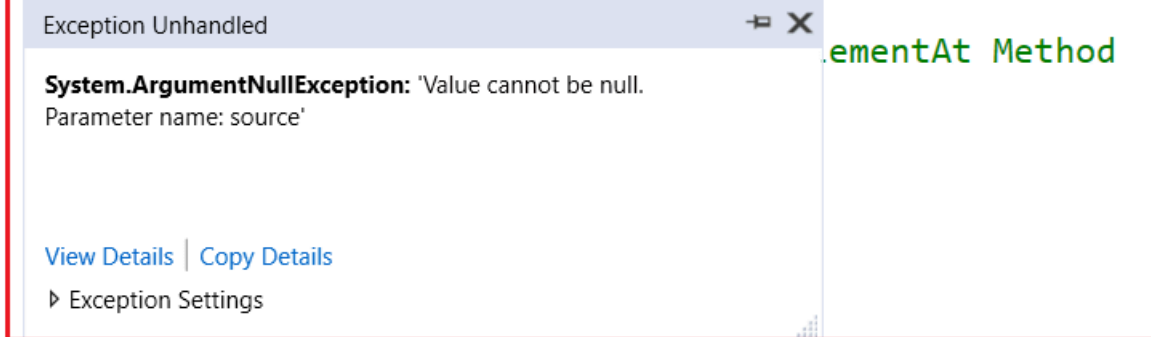
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQElementAtDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source is Null
            List<int> numbers = null;
            //Using ElementAt Method
            int MethodSyntax = numbers.ElementAt(1);
            //Printing the value returned by the ElementAt Method
            Console.WriteLine(MethodSyntax);
            Console.ReadLine();
        }
    }
}

```

```
}  
}
```

Output:

```
//Data Source is Null  
List<int> numbers = null;  
  
//Using ElementAt Method  
int MethodSyntax = numbers.ElementAt(1);
```



Points to Remember:

While working with the LINQ ElementAt Method in C#, we will get runtime **ArgumentOutOfRangeException** exceptions in the following scenarios.

1. If the Data Source is empty. **ArgumentOutOfRangeException**
2. If you specify a negative value for the index position. **ArgumentOutOfRangeException**
3. If you specify the index position which is out of range. **ArgumentOutOfRangeException**

If you don't want the Runtime **ArgumentOutOfRangeException** Exception, instead you want to return a default value, then you need to use the LINQ ElementOrDefault method.

LINQ ElementOrDefault Method in C#:

The LINQ ElementOrDefault method in C# exactly does the same thing as the LINQ ElementAt method except that this method does not throw an **ArgumentOutOfRangeException** exception when the data source is empty or when the supplied index value is out of range or when you specify a negative value for the index position. In such cases, it will return the default value based on the data type of the element the data source contains. If the Data Source is Null, then it will throw ArgumentNullException.

That means this method returns the default value (based on the data source data type) if the index is outside the bounds of the source sequence; otherwise, the element is at the specified position in the source sequence. If you go to the definition of the ElementOrDefault method, then you will see the following signature.

```
public static TSource ElementAtOrDefault<TSource>(this IEnumerable<TSource> source,
int index);
```

- **Purpose:** Returns the element at a specified index in the sequence or the default value for the type if the index is out of range.
- **Usage:** You use ElementAtOrDefault when the index might be outside the bounds of the collection and you want to avoid throwing an exception. The default value for reference types is null, and for value types, it's the default constructor value (for example, 0 for int, false for bool, etc.).

Note: Like the LINQ ElementAt method, the ElementAtOrDefault method also does not have an overloaded version. Let us understand this method with Examples.

Example to Understand LINQ ElementAtOrDefault Method in C#.

Let us see an example to Understand the LINQ ElementAtOrDefault Method in C# with both Method and Query Syntax. Our requirement is to fetch the Element Present in Index Position 1 using the ElementAtOrDefault Method. For a better understanding, please have a look at the following example. Here, we have created one data source which contains integer numbers. Then, we fetch the element present in index position 1 by using the ElementAtOrDefault method, and to that method, we pass the value 1. There is no such operator called ElementAtOrDefault available to write the Query Syntax. If you want, you can combine the method syntax and query syntax to write the code, as shown in the example below.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQElementAtOrDefaultDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            //Method Syntax
            int MethodSyntax = numbers.ElementAtOrDefault(1);
            //Query Syntax
            int QuerySyntax = (from num in numbers
                               select num).ElementAtOrDefault(1);
            Console.WriteLine(MethodSyntax);
            Console.ReadLine();
        }
    }
}
```

Output: 2

What happens if the Index Value is out of the Range of the Collection?

Please have a look at the following example for a better understanding. The data source or collection contains 10 elements so that the index will start from 0 to 9. Let's see what happens when we try to fetch the element from index position 10 or pass a negative index value, as shown in the example below. In this case, as we are using the `ElementAtOrDefault` method, it will not throw any exception. Instead, it will return the default value based on the data type of the data source. Here, the collection data type is `Integer`, and the default value of `Integer` is 0. So, in the below example, it will return 0.

```
using System.Linq;
using System;
using System.Collections.Generic;
namespace LINQElementAtOrDefaultDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<int> numbers = new List<int>() { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
            int MethodSyntax1 = numbers.ElementAtOrDefault(10);
            Console.WriteLine($"Value at Index Position 10: {MethodSyntax1}");
            int MethodSyntax2 = numbers.ElementAtOrDefault(-1);
            Console.WriteLine($"Value at Index Position -1: {MethodSyntax2}");
            Console.ReadLine();
        }
    }
}
```

Output:

```
Value at Index Position 10: 0
Value at Index Position -1: 0
```

LINQ `ElementAt` and `ElementAtOrDefault` Methods in C# with Complex Type:

Let us Understand LINQ `ElementAt` and `ElementAtOrDefault` Methods in C# with Complex Type. For this, we are going to use the following `Student` class. So, first, create a class file with the name **Student.cs** and then copy and paste the following code into it. It is a very simple class having 3 properties and one method, which is to return a collection of students.

```
using System.Collections.Generic;
namespace LINQElementAtOrDefaultDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Department { get; set; }
        public static List<Student> GetAllStudents()
        {
            return new List<Student>()
        }
    }
}
```

```

        {
            new Student { ID = 1, Name = "Preety", Department= "IT"},
            new Student { ID = 2, Name = "Priyanka", Department= "HR"},
            new Student { ID = 3, Name = "Anurag", Department= "HR"},
            new Student { ID = 4, Name = "Pranaya", Department= "IT"},
            new Student { ID = 5, Name = "Hina", Department= "IT"}
        };
    }
}

```

Next, modify the Main method of the Program class as follows. In the example below, I show how to use both LINQ ElementAt and ElementAtOrDefault Methods in C# with Complex Type using Method and Query Syntax.

```

using System.Linq;
using System;
namespace LINQElementAtOrDefaultDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //ElementAtOrDefault Method Syntax
            Student ElementAtMS = Student.GetAllStudents().ElementAt(1);
            //ElementAtOrDefault Query Syntax
            Student ElementAtQS = (from student in Student.GetAllStudents()
                                   select student).ElementAt(2);
            //ElementAtOrDefault Method Syntax
            Student ElementAtOrDefaultMS =
Student.GetAllStudents().ElementAtOrDefault(0);
            //ElementAtOrDefault Query Syntax
            Student ElementAtOrDefaultQS = (from student in Student.GetAllStu
                                             select student).ElementAtOrDefault(3);
            Console.WriteLine($"ID: {ElementAtMS.ID}, Name: {ElementAtMS.Name}
Department: {ElementAtMS.Department}");
            Console.ReadLine();
        }
    }
}

```

Output: ID: 2, Name: Priyanka, Department: HR

What is the Difference Between the LINQ ElementAt and ElementAtOrDefault Methods in C#?

Both methods are used to return an element from the specified index of a data source. But if the element is not available at the specified index position, if the data source is empty, or if we specified a negative value for the Index position, then the ElementAt method will throw an **ArgumentOutOfRangeException** exception. At

the same time, the `ElementAtOrDefault` method will not throw an exception. Instead, it returns a default value based on the data source elements' data type.

When to Use Them

- **ElementAt:** When working with a sequence where you're sure the index you're querying will be within bounds, similar to accessing an array by index. This might be when you've already checked the collection size or when dealing with fixed-size collections.
- **ElementAtOrDefault:** When dealing with dynamic collections where you're unsure if the index you're querying will be within bounds, you want to handle cases that aren't safe. This is particularly useful when you don't want to catch an exception for performance reasons or when you're iterating through indices that could be out of bounds.

Use `ElementAt` when you expect the index to be valid and you want to retrieve the element at that index. If the index is out of range, it will throw an exception, which can help you catch unexpected issues.

Use `ElementAtOrDefault` when uncertain about the index's validity or you want to handle out-of-range scenarios gracefully without throwing exceptions. It returns the default value instead of raising an exception.

It's important to note that using these methods on sequences that do not have an indexer and require enumeration (like `IEnumerable` sequences that are not lists or arrays) can result in poor performance if used repeatedly, as the sequence must be enumerated up to the specified index each time. If multiple index-based retrievals are needed for such sequences, it might be more efficient to first convert the sequence to a list or array.

In the next article, I will discuss the [LINQ First and FirstOrDefault Methods in C#](#) with Examples. In this article, I try to explain the **LINQ ElementAt and ElementAtOrDefault Methods in C#** with Examples. I hope you understand the need and use of these two `ElementAt` and `ElementAtOrDefault` Methods in C#.